

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: K.Rama

Roll No.: A24126552087

Section: CSM-B

Date: 06-09-2026

1. TITLE

Design of an Interactive Student Profile Manager Application using HTML, CSS, and JavaScript

2. AIM

To build a dynamic Student Profile Manager web application that allows a user to enter personal and academic details and dynamically display them as a formatted profile card using HTML for structure, CSS for styling, and JavaScript for DOM manipulation and event handling.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how to link external CSS and JavaScript files to an HTML document.
- To model structured data using a JavaScript class and its constructor.
- To dynamically create and append DOM elements using JavaScript.
- To handle user events such as button clicks.
- To perform basic input validation before displaying data on the page.

4. THEORY

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify. Methods like `document.createElement()` and `appendChild()` allow new elements to be created and inserted dynamically without reloading the page. Event listeners, added using `addEventListener()`, let JavaScript respond to user actions such as clicking a button. JavaScript classes provide a clean way to model real-world entities: a constructor function initializes an object's properties from the values passed to it, so each set of student details can be represented as a single Student object. In this application, four input fields let the user enter a name, roll number, department, and CGPA. When the "Display Student Profile" button is clicked, the values are validated, wrapped in a Student object, and rendered as a profile card built entirely from dynamically created elements.

5. ALGORITHM / PROCEDURE

1. Create `index.html` with a container `div` holding a heading, labelled input fields for Name, Roll Number, Department, and CGPA, a "Display Student Profile" button, and an empty `profileContainer` `div`.
2. Link an external `style.css` file and an external `script.js` file.

3. In style.css, style the container, labels, input fields, the button, and the profile card (including a coloured left border and background to highlight it).
4. In script.js, define a Student class whose constructor stores name, rollNo, department, and cgpa as properties.
5. Select the four input fields, the button, and the profileContainer element using getElementById().
6. Attach a click event listener to the "Display Student Profile" button.
7. Inside the listener, read and trim() the values entered in all four input fields.
8. Validate that no field is empty; if any field is empty, show an alert and stop execution using return.
9. Create a new Student object from the entered values using the constructor.
10. Clear any previously displayed profile, then dynamically build a new profile card containing a heading and four paragraphs (Name, Roll No, Department, CGPA) using document.createElement() and appendChild().
11. Append the completed profile card to profileContainer, and test the application in a browser by entering details and clicking the button.

6. CODE EXPLANATION

The application is split into three files. index.html defines the page structure: a container with a heading, four labelled input fields (id "name", "rollNo", "department", "cgpa"), a "Display Student Profile" button, and an empty div with id "profileContainer" that holds the generated profile card. style.css gives the page a card-like appearance: the container has padding, rounded corners, and a shadow; labels are bold; inputs are full-width with a light border; and the profile card uses a soft blue background with a blue left border to make it stand out. script.js contains the application logic: a Student class encapsulates the four properties, and the button's click handler reads and trims each input value, validates that none are empty, and, if valid, constructs a Student object. The handler then clears the profileContainer, builds a new profile-card div with a heading and one paragraph per property using document.createElement() and appendChild(), and finally appends the completed card to the page.

7. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Student Profile Manager</title>

  <!-- Link CSS file -->
  <link rel="stylesheet" href="style.css">
</head>
```

```

<body>

<div class="container">

    <h1>Student Profile Manager</h1>

    <div class="input-section">

        <label for="name">Name</label>
        <input type="text" id="name" placeholder="Enter your name">

        <label for="rollNo">Roll Number</label>
        <input type="text" id="rollNo" placeholder="Enter roll number">

        <label for="department">Department</label>
        <input type="text" id="department" placeholder="Enter department">

        <label for="cgpa">CGPA</label>
        <input type="number" id="cgpa" placeholder="Enter CGPA" step="0.01">

        <button id="displayBtn">Display Student Profile</button>

    </div>

    <!-- Student profile will be created here dynamically -->
    <div id="profileContainer"></div>

</div>

<!-- Link JavaScript file -->
<script src="script.js"></script>

</body>
</html>
style.css
/* Basic page styling */
body {

```

```
font-family: Arial, sans-serif;
background-color: #f2f4f7;
margin: 0;
padding: 20px;
}

/* Main container */
.container {
width: 90%;
max-width: 600px;
margin: auto;
background-color: white;
padding: 25px;
border-radius: 12px;
box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
}

/* Heading */
h1 {
text-align: center;
margin-bottom: 25px;
}

/* Labels */
label {
display: block;
margin-top: 12px;
font-weight: bold;
}

/* Input fields */
input {
width: 100%;
padding: 10px;
margin-top: 5px;
box-sizing: border-box;
border: 1px solid #ccc;
border-radius: 6px;
```

```

}

/* Button */
button {
    width: 100%;
    margin-top: 20px;
    padding: 12px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    font-size: 16px;
}

/* Student profile */
.profile-card {
    margin-top: 25px;
    padding: 20px;
    border-radius: 10px;
    background-color: #eef5ff;
    border-left: 5px solid #3366cc;
}

/* Profile heading */
.profile-card h2 {
    margin-top: 0;
    text-align: center;
}

/* Profile details */
.profile-card p {
    font-size: 16px;
    margin: 10px 0;
}

script.js
// Student class
class Student {

    constructor(name, rollNo, department, cgpa) {

```

```
        this.name = name;
        this.rollNo = rollNo;
        this.department = department;
        this.cgpa = cgpa;
    }
}
```

```
// Select HTML elements using DOM
```

```
const nameInput = document.getElementById("name");
const rollNoInput = document.getElementById("rollNo");
const departmentInput = document.getElementById("department");
const cgpaInput = document.getElementById("cgpa");
```

```
const displayBtn = document.getElementById("displayBtn");
const profileContainer = document.getElementById("profileContainer");
```

```
// Add click event to button
```

```
displayBtn.addEventListener("click", function () {
```

```
    // Get values entered by the user
```

```
    const name = nameInput.value.trim();
    const rollNo = rollNoInput.value.trim();
    const department = departmentInput.value.trim();
    const cgpa = cgpaInput.value.trim();
```

```
    // Check whether all fields are filled
```

```
    if (name === "" || rollNo === "" || department === "" || cgpa === "") {
        alert("Please fill all the fields.");
        return;
    }
}
```

```
// Create Student object
```

```
const student = new Student(
    name,
```

```
rollNo,  
department,  
cgpa  
);  
  
// Clear previous profile  
profileContainer.innerHTML = "";  
  
// Create profile card dynamically  
const profileCard = document.createElement("div");  
profileCard.className = "profile-card";  
  
// Create heading  
const heading = document.createElement("h2");  
heading.textContent = "Student Profile";  
  
// Create student details  
const namePara = document.createElement("p");  
namePara.textContent = "Name : " + student.name;  
  
const rollPara = document.createElement("p");  
rollPara.textContent = "Roll No : " + student.rollNo;  
  
const departmentPara = document.createElement("p");  
departmentPara.textContent = "Department : " + student.department;  
  
const cgpaPara = document.createElement("p");  
cgpaPara.textContent = "CGPA : " + student.cgpa;  
  
// Add elements to profile card  
profileCard.appendChild(heading);  
profileCard.appendChild(namePara);  
profileCard.appendChild(rollPara);
```

```
profileCard.appendChild(departmentPara);
profileCard.appendChild(cgpaPara);
```

```
// Add profile card to webpage
profileContainer.appendChild(profileCard);
});
```

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Load page with all fields empty	Empty form is shown, no profile card displayed	Form displayed correctly, no profile card present	Pass
TC-02	Click "Display Student Profile" with one or more fields empty	Alert "Please fill all the fields." is shown	Alert displayed, no profile card created	Pass
TC-03	Fill all fields with valid data and click "Display Student Profile"	A profile card appears showing Name, Roll No, Department, and CGPA	Profile card displayed with correct details	Pass
TC-04	Change input values and click "Display Student Profile" again	Previous profile card is replaced with the new details	Old card removed, new card shown correctly	Pass
TC-05	Enter a non-numeric value in the CGPA field	Browser number input restricts entry to numeric values	Only numeric CGPA values accepted	Pass

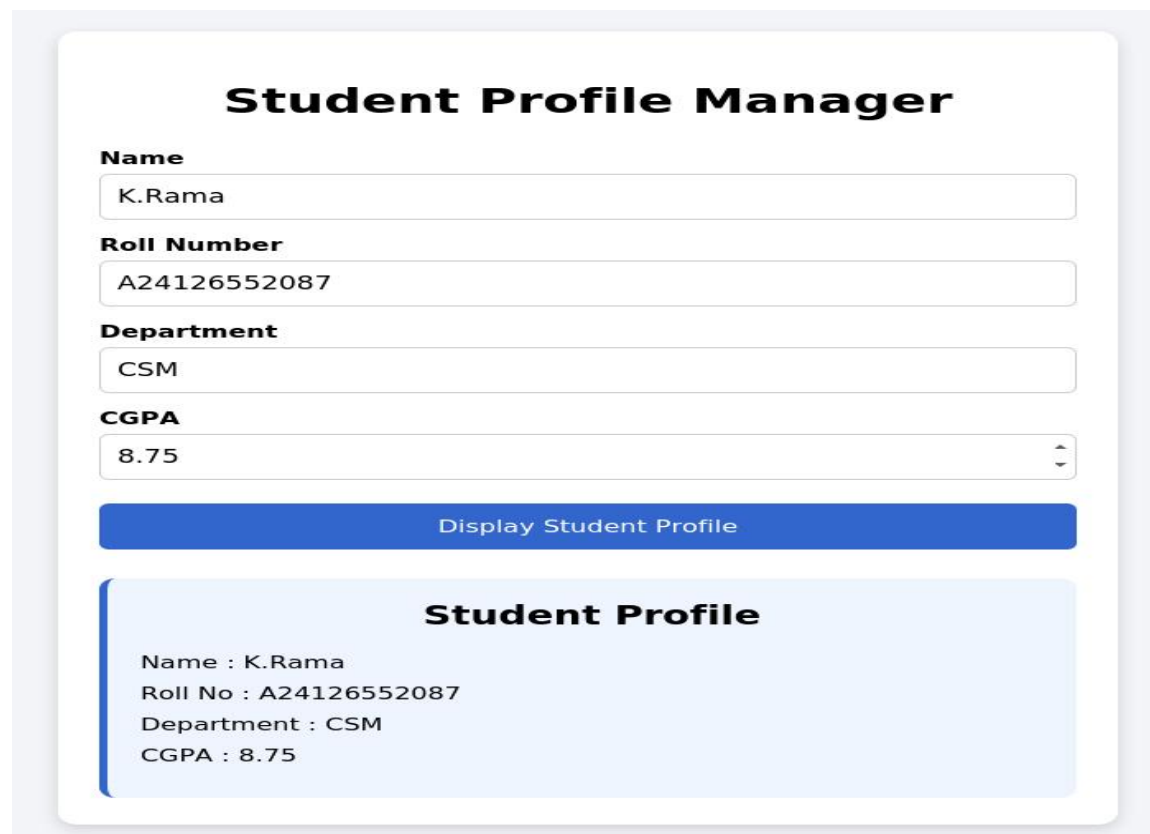
9. OUTPUT

Initial page load (no profile displayed):



The screenshot shows a web form titled "Student Profile Manager". It contains four input fields: "Name" with placeholder text "Enter your name", "Roll Number" with placeholder text "Enter roll number", "Department" with placeholder text "Enter department", and "CGPA" with placeholder text "Enter CGPA". Below these fields is a blue button labeled "Display Student Profile".

After entering student details and clicking "Display Student Profile":



The screenshot shows the same "Student Profile Manager" form, but now with data entered in the fields: "Name" is "K.Rama", "Roll Number" is "A24126552087", "Department" is "CSM", and "CGPA" is "8.75". The blue "Display Student Profile" button is still present. Below the button, a new section titled "Student Profile" is displayed, showing the entered details: "Name : K.Rama", "Roll No : A24126552087", "Department : CSM", and "CGPA : 8.75".

10. RESULT

Thus, an interactive Student Profile Manager web application was successfully designed using HTML, CSS, and JavaScript. The application allows a user to enter personal and academic details, validates the input, and dynamically displays the details as a formatted profile card using a JavaScript class and DOM manipulation. The output was verified in a browser.